VELoop Rewards

Full Stack Developer Internship Task

Project: APPLE WORM ADVENTURE

Full-Stack Puzzle Game + Rewarded-Ad Economy
Reference Game: Snake Puzzle: Slither to Eat
Reference URL:
https://play.google.com/store/apps/details?id=puzzle.snakes.puzzle.slithereat.gaming

Internship Domain: Full Stack Development
Difficulty: Advanced
Project Type: Puzzle Game + Gamification + Ad-Monetization + Reward Economy
Duration: 20 - 24 days from its assigned date

---

1. PROJECT OBJECTIVE

Build a **production-quality full-stack web puzzle game** inspired by the gameplay concept of the provided reference.

The player controls a growing worm/snake, collects apples, navigates obstacles and puzzle structures, and ultimately reaches a destination/portal.

The reference game's central mechanic is simple but strategic:

Move → Eat Apple → Grow → Recalculate Path → Reach Portal.

The reference listing describes a puzzle where each apple increases the snake's length and the player must strategically plan movement to reach the teleportation portal.

However, this internship task has an additional requirement:

THE GAME MUST BE DESIGNED AROUND A SUSTAINABLE AD-FUNDED REWARD ECONOMY.

This is especially important because **VELoop Rewards is a rewards and engagement platform whose monetization depends heavily on advertising.**

Therefore, interns are expected to understand both:

Game Engineering

and

Game Economy Engineering.

The objective is not:

"Show as many ads as possible."

The objective is:

Create meaningful rewarded-ad opportunities that users voluntarily choose because the reward provides genuine gameplay value, while keeping the reward liability economically sustainable against advertising revenue.

---

2. MANDATORY — PLAY THE REFERENCE GAME FIRST

Before coding, install and play the reference game:

Snake Puzzle: Slither to Eat

https://play.google.com/store/apps/details?id=puzzle.snakes.puzzle.slithereat.gaming

Study:

- Main menu
- Level selection
- Snake movement
- Apple collection
- Snake growth
- Obstacles
- Traps
- Portal
- Level completion
- Level failure
- Restart
- Hint system
- Difficulty progression
- UI
- Animations
- Sound
- Reward systems
- Ad placement
- Overall gameplay flow

The reference game specifically describes strategic path planning, apple collection, snake growth and reaching the portal.

---

3. IMPORTANT — DO NOT COPY THE REFERENCE GAME

You are studying the **gameplay concept**, not copying the application.

Do NOT copy:

- Logo
- Name
- Characters
- Snake artwork
- Apple artwork
- UI
- Screenshots
- Level layouts
- Sounds
- Music
- Source code
- APK assets
- Proprietary animations
- Branding

Create your own:

- Game identity
- Snake/worm design
- Apple design
- World
- Obstacles
- UI
- Levels
- Sounds
- Animations
- Reward system

---

4. GAME NAME MUST COME FROM .env

This is a mandatory engineering requirement.

Use:

```
VITE_GAME_NAME=APPLE WORM ADVENTURE
```

The game name must never be hardcoded throughout the application.

---

5. CENTRALIZED GAME CONFIGURATION

Create:

```
src/
└── config/
    └── gameConfig.js
```

Example:

```
export const GAME_NAME =
```

```
  import.meta.env.VITE_GAME_NAME || "APPLE WORM ADVENTURE";
```

Use:

```
import { GAME_NAME } from "../config/gameConfig";
```

Then:

```
<h1>{GAME_NAME}</h1>
```

---

6. GAME NAME MUST BE USED EVERYWHERE

Use `GAME_NAME` for:

- Landing page
- Navbar
- Login
- Registration
- Dashboard
- Level selection
- Game screen
- Game-over screen
- Level completion
- Loading screen
- Leaderboard
- Achievements
- Rewards screen
- Settings
- Profile
- Help
- Footer
- Browser title
- SEO metadata
- Open Graph metadata
- PWA metadata
- Admin panel
- Email templates where applicable

There must be **one source of truth**.

---

7. BACKEND GAME NAME

Backend `.env`:

```
GAME_NAME=APPLE WORM ADVENTURE
```

Backend:

```
const GAME_NAME =
  process.env.GAME_NAME || "APPLE WORM ADVENTURE";
```

Do not hardcode the game name across backend files.

---

8. .env.example

Required:

```
VITE_GAME_NAME=APPLE WORM ADVENTURE
VITE_API_URL=http://localhost:5000/api

GAME_NAME=APPLE WORM ADVENTURE

MONGODB_URI=
JWT_SECRET=
```

Never commit real secrets.

.gitignore:

```
.env
.env.local
.env.production
node_modules
```

---

9. GAME NAME EVALUATION TEST

The evaluator may change:

```
VITE_GAME_NAME=APPLE QUEST
```

or:

```
VITE_GAME_NAME=WORM ESCAPE
```

After rebuild/restart, the new name must automatically appear everywhere.

No component should require manual modification.

---

10. CORE GAMEPLAY

The player controls a worm/snake inside a puzzle environment.

The objective is:

```
Collect apples
    ↓
Grow longer
    ↓
Navigate obstacles
    ↓
```

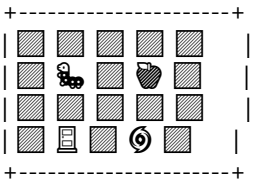
Solve the puzzle
↓
Reach portal
↓
Complete level

---

11. GRID-BASED PUZZLE

Implement a structured game board.

Example:



The board may contain:

- Worm
- Apples
- Portal
- Walls
- Platforms
- Traps
- Bridges
- Moving obstacles
- Special tiles

---

12. WORM MOVEMENT

Support:

Desktop

- Arrow keys
- WASD
- Click controls where appropriate

Mobile

- Swipe
- Direction buttons

The movement must be responsive.

13. WORM GROWTH

Every apple should increase the worm's length.

Example:

Length 2
↓
Eat Apple
↓
Length 3
↓
Eat Apple
↓
Length 4

Growth should affect gameplay.

It must not simply be a visual counter.

---

14. BODY COLLISION

The worm should have actual body collision logic.

Handle:

- Head collision
- Body collision
- Wall collision
- Trap collision
- Invalid movement

If the player moves into its own body:

LEVEL FAILED

unless the current level specifically allows such movement.

---

15. PORTAL SYSTEM

Each level must contain a destination.

Example:

🐛 → 🍏 → 🍏 → 🌀

The player must reach:

🌀 PORTAL

to complete the level.

---

16. PORTAL VALIDATION

Do not allow completion simply because the worm touches the portal.

Validate:

- Correct position
- Required apples collected
- Objective completed
- Worm state
- Level state

---

17. APPLE SYSTEM

Apples can have different types.

Normal Apple

Adds one body segment.

Golden Apple

Adds bonus score.

Time Apple

Adds time if a timed mode exists.

Mystery Apple

Triggers a random predefined effect.

Bonus:

Mega Apple

Adds multiple segments.

---

18. OBSTACLES

Implement multiple obstacle types.

Minimum:

Wall

Blocks movement.

Spike

Causes failure.

Pit

Causes failure.

Locked Gate

Requires an objective.

Moving Block

Changes path.

---

19. ADVANCED OBSTACLES

Bonus:

- Rotating spikes
- Ice tiles
- Teleporters
- One-way gates
- Breakable blocks
- Switches
- Pressure plates
- Bridges
- Moving platforms
- Pushable blocks

---

20. LEVEL OBJECTIVES

Do not make every level identical.

Implement multiple objectives.

Classic

Eat all apples and reach portal.

Collection

Collect a required number of apples.

Length

Reach a specific worm length.

Rescue

Collect special objects.

Key & Gate

Collect key before reaching portal.

Limited Moves

Complete within a move limit.

Perfect Route

Complete without hitting obstacles.

---

21. LEVEL DIFFICULTY

Minimum:

50 PLAYABLE LEVELS

Recommended:

1–10

Tutorial / Beginner

11–20

Easy

21–30

Medium

31–40

Hard

41–50

Expert

Bonus:

100+

200+

levels.

---

22. LEVEL DATA MUST BE DATA-DRIVEN

Do not create 50 separate React pages.

Use structured level data.

Example:

```
{
 id: 1,
 difficulty: "easy",
 grid: {
  rows: 8,
  columns: 8
 },
 wormStart: {
  x: 1,
  y: 6
 },
 apples: [
  { x: 4, y: 3 }
 ],
 portal: {
  x: 7,
  y: 1
 },
 obstacles: [],
 moveLimit: 30,
 objective: "reach_portal"
}
```

23. LEVEL GENERATION

Level layouts may be:

Hand-designed

Recommended for the first 50 levels.

Procedurally generated

Bonus.

or:

Generated levels must be validated to ensure they are actually solvable.

---

24. SOLVABILITY VALIDATION

Every published level must have at least one valid solution.

Create a validation tool/function where practical.

Example:

```
isLevelSolvable(level)
```

This is particularly important if levels are generated automatically.

---

25. LEVEL SELECT

Create a professional level-selection system.

Display:

LEVEL 01 ★★★
LEVEL 02 ★★☆
LEVEL 03 ★☆☆
LEVEL 04 🔒

Track:

- Completion
- Stars
- Best score
- Best moves

26. LEVEL UNLOCKING

Initially:

```
Level 1 = unlocked
```

After completion:

```
Level 2 = unlocked
```

Continue sequentially.

---

27. STAR SYSTEM

Each level should have:

★
★★
★★★

Stars may depend on:

- Moves
- Time
- Apples collected
- Deaths
- Booster usage
- Efficiency

---

28. SCORE SYSTEM

Example:

Apple collected	+10
Golden apple	+50
Level completion	+100
Perfect completion	+250
Low moves bonus	+bonus
No revive bonus	+bonus

Avoid random score generation.

---

29. MOVE COUNTER

Display:

```
Moves: 17
```

Track:

- Valid moves
- Invalid attempts
- Restarts
- Revives
- Hints
- Boosters

30. UNDO SYSTEM

Implement a real undo system.

Undo should restore:

- Worm position
- Worm body
- Apples
- Obstacles
- Move count
- Objective state

Optional economic model:

```
First undo = free
Additional undo = rewarded-ad opportunity
```

This should be controlled by server/configuration rather than hardcoded.

---

31. HINT SYSTEM

Create a hint system.

A hint can:

- Highlight the next direction
- Show a recommended route
- Highlight an apple
- Highlight a dangerous tile
- Show portal path

---

32. REWARDED-AD HINT

One important VELoop monetization opportunity:

💡 GET FREE HINT
Watch a short rewarded ad

After successful ad completion:

```
Hint unlocked
```

The client must NOT simply assume that an ad was watched.

The backend should verify the reward event/token where the chosen ad provider supports server-side verification.

33. REVIVE SYSTEM — CORE ECONOMIC FEATURE

When the player fails:

LEVEL FAILED

Show:

♡ REVIVE

Watch Ad
Continue from your last safe state

[WATCH AD & REVIVE]

[RESTART]

---

34. REVIVE MECHANIC

A successful rewarded ad can restore the player's last valid game state.

Example:

Death
↓
Save checkpoint
↓
Show Revive
↓
Rewarded Ad
↓
Verify reward
↓
Restore checkpoint
↓
Continue

---

35. REVIVE LIMIT

Do NOT allow unlimited revives.

Example configuration:

MAX_REVIVES_PER_LEVEL = 1;

or:

1 free revive
+
1 rewarded revive

The exact value should be configurable.

---

36. REVIVE ECONOMICS

Revive is not just a UI feature.

Track:

reviveRequested
reviveCompleted
reviveRewardGranted
reviveRejected

This allows VELoop to measure:

Ad Opportunity
→ Ad Completion
→ Gameplay Continuation
→ Session Length

---

37. DOUBLE SCORE — REWARDED AD

After completing a level:

LEVEL COMPLETE

Score: 4,500

[CLAIM 4,500]
[WATCH AD → 2X SCORE]

If the user chooses:

WATCH AD → 2X SCORE

and the ad is successfully completed:

Final Score = 9,000

The multiplier must be applied server-side.

---

38. DOUBLE COINS — REWARDED AD

After completion:

You earned:

100 Coins

Show:

[CLAIM 100]
[WATCH AD → GET 200]

After verified rewarded-ad completion:

Reward = 200 Coins

---

39. IMPORTANT — NO FAKE AD REWARD

Do NOT implement:

onClick={() => coins += 200}

when the user presses the button.

Correct flow:

User requests reward
↓
Ad starts
↓
Ad completes
↓
Provider reward callback / verification
↓
Backend validates
↓
Backend grants reward
↓
Transaction recorded
↓
Frontend updates wallet

40. EXTRA MOVES — REWARDED AD

When the player reaches a move limit:

NO MOVES LEFT

Offer:

🎯 GET +5 MOVES

Watch Rewarded Ad

After successful verification:

Moves +5

Limit this reward to prevent economic abuse.

---

41. EXTRA LIFE — REWARDED AD

Optional:

♡ +1 LIFE
Watch Ad

Only use where the selected game mode actually has lives.

---

42. BOOSTER — REWARDED AD

Example:

MAGIC BOOSTER

Watch Ad to use this booster.

The reward may unlock:

- Undo
- Hint
- Safe Move
- Extra Move
- Remove Obstacle

43. ECONOMICALLY STABLE REWARD SYSTEM

This is a **mandatory business requirement**.

The intern must understand:

Every reward has an economic cost.

For example:

```
Ad Revenue
    ↓
Platform Revenue
    ↓
Reward Budget
    ↓
User Reward
```

The reward system must not blindly distribute unlimited coins.

---

44. REWARD ECONOMY CONFIGURATION

Create a centralized economy configuration.

Example:

```
export const ECONOMY_CONFIG = {
  levelCompletionCoins: 25,

  rewardedAd: {
    doubleCoinsMultiplier: 2,
    doubleScoreMultiplier: 2,
    reviveEnabled: true,
    extraMoves: 5,
    hintEnabled: true,

    maxRewardedAdsPerDay: 10,
    maxRevivesPerLevel: 1,
    maxDoubleScorePerLevel: 1,
    maxDoubleCoinsPerLevel: 1
  }
};
```

Do not scatter these values across components.

45. ECONOMY CONFIG SHOULD BE SERVER CONTROLLED

For a production architecture, important economic parameters should be configurable from the backend.

Example:

GET /api/economy/config

The server may return:

```
{
  "levelCompletionCoins": 25,
  "doubleScoreEnabled": true,
  "doubleCoinsEnabled": true,
  "reviveEnabled": true,
  "extraMoves": 5,
  "dailyRewardedAdCap": 10
}
```

This allows the business to change the economy without rebuilding the frontend.

---

46. REWARD LEDGER

Create a proper reward transaction system.

Collection:

rewardTransactions

Each transaction should contain:

userId
gameId
gameSessionId
levelId
rewardType
amount
multiplier
source
adProvider
adEventId
status
createdAt

Example:

userId: 123
rewardType: DOUBLE_COINS
amount: 200
source: REWARDED_AD
status: VERIFIED

47. IDEMPOTENCY

This is mandatory for reward transactions.

If the same ad reward callback is received twice:

AD_EVENT_123

the user must receive the reward only once.

Use:

unique adEventId

or equivalent idempotency logic.

---

48. REWARD STATES

Every rewarded action should have a state.

REQUESTED
AD_STARTED
AD_COMPLETED
VERIFYING
VERIFIED
GRANTED
REJECTED
EXPIRED

This makes the system auditable.

---

49. AD OPPORTUNITY MODEL

Track ad opportunities.

Example:

adOpportunities

Fields:

userId
sessionId
levelId
type
createdAt
expiresAt
status

Types:

REVIVE
DOUBLE_SCORE
DOUBLE_COINS
EXTRA_MOVES
HINT
BOOSTER

---

50. AD PLACEMENT STRATEGY

The project should prioritize **rewarded ads**, not aggressive interruption.

Preferred placements:

1. Revive

After failure.

2. Double Score

After completion.

3. Double Coins

After completion.

4. Extra Moves

When stuck/limit reached.

5. Hint

When requested.

6. Booster

When requested.

---

51. INTERSTITIAL ADS

If implemented, use them sparingly.

Do NOT show an interstitial:

- Immediately after every click
- During active gameplay
- During a puzzle solution
- Before every restart
- Immediately after a rewarded ad
- Multiple times in a short period

Use a configurable cooldown.

Example:

```
INTERSTITIAL_COOLDOWN_SECONDS = 120;
```

---

52. NO AD SPAM

This is particularly important.

The reference game's current listing includes ads, and user reviews have criticized excessive/interruption-heavy ad frequency.

Your implementation should therefore prioritize:

Opt-in value-based rewarded ads over disruptive advertising.

The goal is to increase:

- Session length
- Engagement
- Rewarded-ad completion
- Retention
- Revenue

without destroying gameplay.

---

53. AD FREQUENCY CONTROL

Create configurable rules:

max rewarded ads per session
max rewarded ads per hour
max rewarded ads per day
minimum time between ads
maximum revives
maximum double rewards

Example:

```
const AD_LIMITS = {
  maxRewardedPerSession: 5,
  maxRewardedPerHour: 8,
  maxRewardedPerDay: 15,
  minSecondsBetweenRewardedAds: 30
};
```

These are examples only and should be tuned using actual revenue/engagement data.

---

54. REWARD CAP

Never allow:

Infinite coins
Infinite score multiplier
Infinite revives
Infinite boosters

Every reward type must have a configurable cap.

55. SCORE VS COINS

Keep these concepts separate.

Score

Used for:

- Leaderboards
- Level performance
- Personal best

Coins

Used for:

- Hints
- Boosters
- Cosmetics
- Optional game utilities

Do not let users manipulate leaderboard score through client-side coin transactions.

---

56. COIN ECONOMY

Example:

Level Completion	+25	Daily Challenge	+100
Golden Apple	+10	Rewarded Double Coins	×2
3-Star Bonus	+20		

Coin spending:

Hint	-50	Extra Undo	-40
Booster	-100	Cosmetic	-500

These values must be configurable.

---

57. REWARD BUDGET MODEL

Create an economy model based on:

```
Expected Ad Revenue
      ↓
Reward Allocation
      ↓
Maximum Reward Liability
```

For example, conceptually:

```
Ad Revenue per 1,000 impressions
            ↓
```

```
Available reward budget
        ↓
Maximum coins distributed
```

The intern does NOT need to know the exact commercial ad-network payout.

Instead, build the architecture so that reward values can be changed centrally after actual monetization data is available.

---

58. ECONOMY SAFETY RULE

The game must never assume:

```
1 ad = unlimited reward
```

Instead:

```
Verified Ad Event
+
Eligible Reward
+
Daily Cap
+
Session Cap
+
Idempotency
=
Reward Granted
```

---

59. SERVER-SIDE ECONOMY

Important reward operations should go through the backend.

Example:

```
POST /api/rewards/ad/request
```

Then:

```
POST /api/rewards/ad/verify
```

Then:

```
POST /api/rewards/ad/claim
```

The backend determines whether the reward is actually allowed.

60. REWARD API

Recommended:

POST /api/rewards/ad/opportunity
POST /api/rewards/ad/verify
POST /api/rewards/ad/claim
GET /api/rewards/balance
GET /api/rewards/history
GET /api/economy/config

---

61. REVIVE API

POST /api/games/:sessionId/revive/request
POST /api/games/:sessionId/revive/verify

The server should verify:

- Session ownership
- Level
- Current game state
- Revive count
- Ad verification
- Eligibility

---

62. DOUBLE SCORE API

POST /api/games/:sessionId/double-score/request
POST /api/games/:sessionId/double-score/verify

The final score must be calculated server-side.

---

63. DOUBLE COINS API

POST /api/games/:sessionId/double-coins/request
POST /api/games/:sessionId/double-coins/verify

Do not trust:

```
{
  "coins": 5000
}
```

from the browser.

The backend determines the actual reward.

64. EXTRA MOVES API

POST /api/games/:sessionId/extra-moves/request
POST /api/games/:sessionId/extra-moves/verify

---

65. AD PROVIDER ABSTRACTION

Do not tightly couple the entire game to one ad provider.

Create:

```
services/
└── ads/
    ├── adService.js
    ├── rewardedAdService.js
    └── adProviderAdapter.js
```

Conceptually:

```
showRewardedAd({
  placement: "REVIVE"
});
```

This allows the platform to change providers later.

---

66. AD PLACEMENTS

Define named placements:

REVIVE
DOUBLE_SCORE
DOUBLE_COINS
EXTRA_MOVES
HINT
BOOSTER
DAILY_REWARD

Do not use arbitrary strings everywhere.

---

67. REWARDED-AD FLOW

Required architecture:

Player clicks:
WATCH AD & REVIVE

↓

Create ad opportunity

↓

Ad provider starts

↓

Player completes ad

↓

Reward verification

↓

Backend verifies eligibility

↓

Reward ledger transaction

↓

Grant reward

↓

Update game state

---

68. FAILED AD

If an ad fails to load:

Unable to load reward ad.
Please try again.

Do not deduct coins.

Do not consume the revive.

Do not grant the reward.

---

69. USER CLOSES AD EARLY

If the provider reports that the rewarded ad was not completed:

Reward not completed.

The reward must not be granted.

70. DUPLICATE CALLBACK

If the provider sends the same reward event twice:

```
Event ID = ABC123
```

the second request must be rejected as already processed.

---

71. REWARD HISTORY

Users should be able to see:

```
Reward History

+---------------------------+
| Double Coins     +200     |
| Rewarded Ad      ✓        |
| Today                     |
+---------------------------+

| Revive           Used     |
| Rewarded Ad      ✓        |
+---------------------------+
```

---

72. PLAYER WALLET

Create a simple in-game wallet.

Display:

```
  2,450
```

The wallet should be backed by the database.

---

73. WALLET LEDGER

Do not simply update:

```
user.coins += 100;
```

without a transaction record.

Record:

CREDIT
DEBIT
REVERSAL
BONUS
AD_REWARD
LEVEL_REWARD
BOOSTER_PURCHASE

---

74. ECONOMIC AUDIT

Admin should be able to inspect:

Total ad opportunities
Completed ads
Failed ads
Rewarded ads
Coins granted
Coins spent
Revives
Double-score uses
Double-coin uses
Average rewards per user

---

75. ADMIN ECONOMY DASHBOARD — BONUS/STRONGLY RECOMMENDED

Create:

Admin → Economy

Display:

Today's Ad Opportunities
Today's Completed Rewarded Ads
Today's Coins Granted
Today's Coins Spent
Rewarded Ad Completion Rate
Average Ads/User
Average Reward/User

This is extremely useful for balancing the VELoop economy.

---

76. ECONOMY ANALYTICS

Track events:

game_started
level_started
level_completed
level_failed
revive_requested
revive_completed
double_score_requested
double_score_completed
double_coins_requested
double_coins_completed

extra_moves_requested
extra_moves_completed
hint_requested
hint_completed
booster_used
reward_granted
reward_rejected

---

77. ECONOMY METRICS

The system should allow analysis of:

Rewarded Ad Completion Rate

completed ads / ad opportunities

Reward Cost

total reward units granted

Engagement

sessions / user

Revive Conversion

revive ads completed / revive opportunities

Double Score Conversion

double-score completions / offers

---

78. GAMEPLAY SHOULD NEVER REQUIRE ADS

A player should still be able to play the core game without watching an advertisement.

Ads provide:

- Optional convenience
- Bonus rewards
- Recovery
- Extra progression

They should not make normal gameplay impossible.

---

79. REWARDED ADS SHOULD FEEL LIKE A CHOICE

Good:

LEVEL FAILED

Continue your current progress?

[WATCH AD & REVIVE]

[RESTART]

Bad:

WATCH THIS AD

without a meaningful choice.

---

80. AUDIO — MANDATORY

Implement:

- Button click
- Worm movement
- Apple collection
- Portal
- Failure
- Revive
- Booster
- Hint
- Coin reward
- Double reward
- Level complete
- Star award
- Achievement

---

81. BACKGROUND MUSIC

Provide:

🎵 Music ON/OFF
🔊 Sound ON/OFF

Persist preferences.

---

82. ANIMATIONS

Required:

Worm movement

Smooth movement between tiles.

Growth

Body segment animation.

Apple collection

Pop/scale effect.

Portal

Glow/pulse.

Level complete

Celebration.

Coins

Coin collection animation.

Revive

Restoration animation.

Double reward

Reward multiplication animation.

---

83. PARTICLE EFFECTS

Use particles for:

- Apple collection
- Golden apple
- Portal
- Level completion
- Coins
- Revive
- Achievement

Optimize them for mobile.

---

84. AUTHENTICATION

Implement:

POST /api/auth/register
POST /api/auth/login
GET /api/auth/me
POST /api/auth/logout

Use:

- JWT
- bcrypt
- Protected routes
- Secure cookies where appropriate

---

85. PLAYER DASHBOARD

Display:

Current Level
Completed Levels
Total Stars
Total Score
Coins
Apples Collected
Best Streak
Achievements
Rewarded Ads Watched

Example:

LEVEL 34

☆ 82 Stars

245,820 Score

1,482 Apples

3,420 Coins

27 Rewarded Ads

---

86. GAME HISTORY

Store:

Level
Score
Moves
Stars
Apples
Revives
Boosters
Coins Earned
Completed
Date

---

87. GAME SESSION

When a level starts:

POST /api/games/start

Store:

sessionId
userId
levelId
startTime
initialState
status

---

88. CHECKPOINT SYSTEM

For revive functionality, the server/session should maintain a valid checkpoint.

Example:

checkpointState
checkpointMove
checkpointScore
checkpointTimestamp

Do not trust arbitrary client-provided checkpoint data.

89. SERVER-SIDE SCORE VALIDATION

The backend should validate:

- Level ID
- Session ownership

- Move count
- Apples collected
- Completion state
- Score
- Booster usage
- Revive count

The browser must never be the sole authority.

---

90. ANTI-CHEAT

Detect:

- Impossible scores
- Impossible completion times
- Impossible movement counts
- Duplicate completion
- Reused sessions
- Fake reward requests
- Duplicate ad events
- Invalid reward amounts
- Modified level IDs
- Replay attacks

---

91. DATABASE COLLECTIONS

Recommended:

users
levels
gameSessions
playerProgress
rewardTransactions
walletTransactions
adOpportunities
achievements
userAchievements
economyConfigs
dailyChallenges

---

92. USER MODEL

```
{
 name,
 email,
 passwordHash,

 currentLevel,
 totalScore,
 totalStars,
 coins,

 applesCollected,
 completedLevels,

 rewardedAdsWatched,
```

createdAt,
updatedAt
}

---

93. REWARD TRANSACTION MODEL

{
userId,
gameId,
sessionId,

type,
source,

amount,
multiplier,

adProvider,
adEventId,

status,

createdAt
}

---

94. AD OPPORTUNITY MODEL

{
userId,
sessionId,
levelId,

placement,

status,

expiresAt,

adProvider,
adEventId,

createdAt
}

95. WALLET TRANSACTION MODEL

{
userId,

```
  type,
  amount,

  balanceBefore,
  balanceAfter,

  referenceId,
  source,

  createdAt
}
```

---

96. LEADERBOARD

Create:

Global Leaderboard

Rank
Player
Score
Stars
Levels

Personal Rank

Your Rank
Your Score

Optional:

- Weekly
- Monthly
- Friends

---

97. ACHIEVEMENTS

Implement at least 10.

Examples:

First Apple

Collect first apple.

Worm Beginner

Complete 5 levels.

Portal Finder

Complete 10 levels.

Perfect Route

Complete a level without failure.

Apple Collector

Collect 500 apples.

Master Solver

Complete 25 levels.

Worm Legend

Complete 50 levels.

Star Collector

Earn 100 stars.

Ad Smart

Use a rewarded recovery feature.

Economy Explorer

Use multiple reward features.

---

98. DAILY CHALLENGE — BONUS

Create a daily puzzle.

Example:

DAILY APPLE CHALLENGE

Objective:
Collect 5 apples

Moves:
20

Reward:
100 Coins

Optional:

WATCH AD → 2X DAILY REWARD

Again, the reward must be server-validated and capped.

---

99. STREAK SYSTEM — BONUS

Track:

Daily Streak

Example:

🔥 7 DAYS

Daily rewards:

Day 1 → 25 coins
Day 2 → 30 coins
Day 3 → 40 coins
...

Avoid exponential rewards that can destabilize the economy.

---

100. OFFLINE MODE — BONUS

The reference game is positioned as a casual puzzle experience, and similar Apple Worm titles emphasize offline play.

A web implementation may support:

- PWA
- Cached levels
- Local gameplay
- IndexedDB

However:

Rewarded-ad rewards and leaderboard submissions must require server connectivity and verification.

Never grant server-backed rewards solely from offline state.

---

101. UI PAGES

Minimum:

/
/login
/register
/dashboard
/levels
/game/:levelId
/leaderboard
/achievements
/rewards
/wallet
/profile
/settings
/help

Admin:

/admin
/admin/economy
/admin/rewards
/admin/analytics

102. LANDING PAGE

Example:

{GAME_NAME}

EAT.
GROW.

Guide your worm.
Collect every apple.

SOLVE.

Find the perfect path.

[PLAY NOW]

Use:

{GAME_NAME}

---

103. REWARD CENTER

Create a dedicated:

 Reward Center

Example:

Today's Opportunities

♡ Revive
Watch Ad

Double Coins
Watch Ad

🏆 Double Score
Watch Ad

💡 Free Hint
Watch Ad

🎯 +5 Moves
Watch Ad

The Reward Center should clearly show eligibility/cooldowns.

---

104. REWARD BUTTON STATES

Example:

AVAILABLE
COOLDOWN
Try again in 00:42
LIMIT REACHED
AD UNAVAILABLE
COMPLETED

Do not show an active reward button when the user is not eligible.

---

105. REWARD CLAIM UI

After successful ad verification:

🎉 REWARD CLAIMED!

2,850

+200 Coins

Use an animation.

New Balance:

---

106. GAME ENGINE ARCHITECTURE

Recommended:

```
src/
├── components/
│   ├── Worm/
│   ├── Apple/
│   ├── Portal/
│   ├── GameBoard/
│   ├── GameHUD/
│   ├── RewardModal/
│   ├── ReviveModal/
│   └── BoosterBar/
│
├── game/
│   ├── gameEngine.js
│   ├── movement.js
│   ├── collision.js
│   ├── objectiveManager.js
│   ├── levelManager.js
│   ├── checkpointManager.js
│   └── scoreManager.js
│
├── economy/
│   ├── economyConfig.js
│   ├── rewardTypes.js
│   └── rewardUtils.js
│
├── services/
│   ├── authService.js
│   ├── gameService.js
│   ├── rewardService.js
│   ├── walletService.js
│   └── adService.js
│
├── config/
│   └── gameConfig.js
│
├── data/
│   └── levels/
│
├── pages/
```

```
│
├── hooks/
│
└── utils/
```

---

107. GAME STATE MACHINE

Use clear states:

LOADING
READY
PLAYING
MOVING
APPLE_COLLECTED
FAILED
REVIVE_PENDING
REVIVED
COMPLETED
REWARD_PENDING
PAUSED

This prevents conflicting gameplay/reward actions.

---

108. REWARD STATE MACHINE

Use:

OFFERED
REQUESTED
AD_STARTED
AD_COMPLETED
VERIFYING
VERIFIED
GRANTED
REJECTED
EXPIRED

Do not mix reward state with ordinary UI state.

---

109. TECHNOLOGY STACK

SUGGESTED STACK

Frontend

React.js
Vite
Tailwind CSS
JavaScript / TypeScript

Backend

Node.js
Express.js

Database

MongoDB
MongoDB Atlas

Authentication

JWT

bcrypt

Game Rendering

Choose an appropriate technology:

HTML/CSS
SVG
Canvas
Phaser
PixiJS

---

110. MERN IS THE SUGGESTED STACK

The preferred/suggested architecture for this assignment is:

MongoDB
Express.js
React.js
Node.js

However:

INTERNS ARE ALLOWED TO USE THEIR PREFERRED TECH STACK.

For example:

Next.js
PostgreSQL
Supabase
Firebase
Django
FastAPI
Spring Boot
Laravel
NestJS
etc.

provided that:

1. The complete functionality is implemented.
2. Authentication works.
3. Database persistence works.
4. Reward economy works.
5. Ad reward verification works.
6. Server-side validation exists.
7. The project is properly documented.
8. The intern explains why they selected the alternative stack.

The stack is flexible.

The **engineering quality is not.**

111. AD PROVIDER FLEXIBILITY

The intern may use an appropriate web-supported advertising solution or create a provider abstraction/mock integration where the actual provider cannot be used in the local environment.

The important requirement is the architecture:

```
Game
↓
Ad Placement
↓
Ad Provider
↓
Reward Verification
↓
Backend
↓
Economy Ledger
↓
Reward
```

Do not hardcode a fake:

```
adFinished = true
```

and claim it is an ad integration.

---

112. GITHUB REQUIREMENTS

Repository:

```
apple-worm-adventure/
│
├── client/
├── server/
├── README.md
├── .gitignore
└── .env.example
```

Do not upload:

.env	JWT secrets
node_modules	MongoDB credentials
API keys	ad-network secrets

---

113. README REQUIREMENTS

README must explain:

1. Project overview
2. Reference game

3. Gameplay
4. Game architecture
5. Level system
6. Economy system
7. Rewarded ads
8. Ad placements
9. Reward verification
10. Wallet
11. Database
12. API
13. Authentication
14. Anti-cheat
15. Environment variables
16. Deployment
17. Screenshots
18. Demo video
19. Known limitations

---

114. ECONOMY DOCUMENTATION — MANDATORY

README must include an:

Economy Design

Explain:

How users earn

Level rewards
Daily rewards
Verified rewarded ads
Achievements
Challenges

How users spend

Hints
Boosters
Extra utilities
Cosmetics

How advertising supports rewards

Rewarded ad
↓
Ad monetization
↓
Verified event
↓
Controlled reward
↓
User engagement

115. ECONOMIC SAFETY DOCUMENTATION

The intern must explain:

- Daily reward cap
- Session reward cap
- Revive cap
- Double-score cap

- Double-coins cap
- Ad cooldown
- Duplicate reward protection
- Server validation
- Wallet ledger
- Reward transaction idempotency

---

116. DEPLOYMENT

Submit:

Frontend:
https://your-project.vercel.app

Backend:
https://your-api.onrender.com

GitHub:
https://github.com/username/repository

The application must actually work.

---

117. DEMO VIDEO

The demo MUST show:

Gameplay

1. Landing page
2. Login
3. Dashboard
4. Level selection
5. Worm movement
6. Apple collection
7. Worm growth
8. Obstacles
9. Portal
10. Level completion
11. Level failure

Economy

12. Wallet
13. Revive offer
14. Rewarded-ad flow
15. Double Score
16. Double Coins
17. Extra Moves
18. Hint reward
19. Reward history

Backend

20. Game session
21. Progress persistence
22. Database
23. Leaderboard
24. Server validation

Responsive

25. Mobile gameplay 26. Desktop gameplay

---

118. MANDATORY FEATURES

The following are compulsory:

Game

- [ ] Play reference game
- [ ] Original game identity
- [ ] .env game name
- [ ] Centralized GAME_NAME
- [ ] Game name used everywhere
- [ ] Grid puzzle
- [ ] Worm movement
- [ ] Apple collection
- [ ] Worm growth
- [ ] Body collision
- [ ] Obstacles
- [ ] Portal
- [ ] Multiple objectives
- [ ] 50+ levels
- [ ] Level progression
- [ ] Level unlocking
- [ ] Star system
- [ ] Score
- [ ] Move tracking
- [ ] Restart
- [ ] Hint
- [ ] Animations
- [ ] Sound
- [ ] Background music
- [ ] Responsive design
- [ ] Touch controls

Full Stack

- [ ] Authentication
- [ ] Dashboard
- [ ] Database
- [ ] Game sessions
- [ ] Player progress
- [ ] Game history
- [ ] Leaderboard
- [ ] Achievements
- [ ] Server-side validation
- [ ] Anti-cheat basics

VELoop Economy

- [ ] Wallet
- [ ] Wallet ledger
- [ ] Reward transaction ledger
- [ ] Ad opportunity tracking
- [ ] Reward verification
- [ ] Reward idempotency
- [ ] Revive via rewarded ad
- [ ] Double Score via rewarded ad
- [ ] Double Coins via rewarded ad
- [ ] Extra Moves via rewarded ad
- [ ] Hint via rewarded ad
- [ ] Booster via rewarded ad
- [ ] Reward caps
- [ ] Ad cooldown
- [ ] Daily reward limits
- [ ] Reward history
- [ ] Economy configuration
- [ ] Server-side reward calculation

Deployment

- [ ] GitHub
- [ ] Frontend deployment
- [ ] Backend deployment

- [ ] .env.example
- [ ] README
- [ ] Demo video

---

119. BONUS FEATURES

Extra credit:

- [ ] Daily challenge
- [ ] Daily streak
- [ ] 100+ levels
- [ ] Procedural level generation
- [ ] Level editor
- [ ] Admin economy dashboard
- [ ] Admin reward controls
- [ ] Weekly leaderboard
- [ ] Monthly leaderboard
- [ ] Friends leaderboard
- [ ] PWA
- [ ] Offline gameplay
- [ ] Cloud save
- [ ] Multiple worms
- [ ] Skins
- [ ] Cosmetic shop
- [ ] Seasonal events
- [ ] Advanced analytics
- [ ] A/B testing for reward values
- [ ] Remote economy configuration
- [ ] Multiple ad providers
- [ ] Reward fraud detection
- [ ] Advanced anti-cheat

120. ECONOMIC QUALITY BAR

This section is especially important.

The intern must understand:

VELoop is not simply giving away coins.

Coins have an economic value within the platform.

Therefore:

```
USER ACTION
   ↓
ENGAGEMENT
   ↓
AD OPPORTUNITY
   ↓
AD REVENUE
   ↓
CONTROLLED REWARD
   ↓
USER RETENTION
```

The system should be designed so that rewards are:

- Useful
- Attractive
- Controlled
- Measurable
- Auditable
- Sustainable

---

121. BAD ECONOMY EXAMPLE

Do NOT build:

Every ad = 10,000 coins

or:

Unlimited revive

or:

Unlimited double coins

or:

Client says ad completed → give reward

or:

Coins are only stored in localStorage

These approaches are unacceptable.

122. GOOD ECONOMY EXAMPLE

A better design:

Level Complete
↓
25 Coins
↓
Optional:
Watch Rewarded Ad
↓
Verified
↓
50 Coins
↓
Daily Cap Checked
↓
Ledger Transaction
↓
Wallet Updated

This provides a meaningful reward without creating uncontrolled liabilities.

---

123. REWARD CONFIGURATION MUST BE CHANGEABLE

For example:

```
{
  levelReward: 25,

  rewardedAds: {
    doubleCoins: true,
    doubleScore: true,
    revive: true,
    extraMoves: 5,

    dailyCap: 10,
    sessionCap: 5,

    maxRevivesPerLevel: 1
  }
}
```

If the business later changes:

25 coins → 15 coins

the entire game should not require rewriting gameplay components.

124. REMOTE ECONOMY — ADVANCED

An excellent implementation will allow an admin to change:

Level reward
Double coin multiplier
Extra moves
Revive availability
Daily ad cap
Reward cooldown
Booster cost
Hint cost

without modifying frontend code.

---

125. FINAL EVALUATION CRITERIA

Gameplay — 20%

- Movement
- Apple mechanics
- Growth
- Obstacles
- Portal
- Level design

Game Engineering — 15%

- State management
- Collision
- Level validation
- Architecture
- Performance

Full Stack — 15%

- Authentication
- APIs
- Database
- Progress
- Game sessions

VELoop Economy — 25%

- Rewarded ads
- Revive
- Double score
- Double coins
- Extra moves
- Wallet
- Ledger
- Reward validation
- Caps
- Anti-abuse
- Economy configuration

This section carries the highest weight because the project is being developed for a rewards/engagement platform.

UI/UX — 10%

- Responsive
- Mobile controls
- Animations
- Visual quality

Audio/Feedback — 5%

- Sound
- Music
- Effects

Documentation — 5%

- README
- Architecture
- Economy documentation
- Deployment

Code Quality — 5%

- Clean code
- Security
- Reusability
- Maintainability

---

126. FINAL SUBMISSION FORMAT

Intern Name:
Intern ID:

Project Name:
Configured Game Name:

GitHub:
[URL]

Frontend:
[URL]

Backend:
[URL]

Demo Video:
[URL]

Number of Levels:
XX

Tech Stack:
MERN / Other

Authentication:
Implemented / Not Implemented

Database:
MongoDB / Other

Rewarded Ads:
Implemented / Mocked / Provider

Ad Placements:
[List]

Wallet:
Implemented / Not Implemented

Reward Ledger:
Implemented / Not Implemented

Revive:
Implemented / Not Implemented

Double Score:
Implemented / Not Implemented

Double Coins:
Implemented / Not Implemented

Extra Moves:
Implemented / Not Implemented

Hint:
Implemented / Not Implemented

Leaderboard:
Implemented / Not Implemented

Achievements:
Implemented / Not Implemented

Admin Economy:
Implemented / Not Implemented

Bonus Features:
[List]

Known Limitations:
[List]

---

127. FINAL INSTRUCTION

DO NOT START CODING IMMEDIATELY.

First, play:

Snake Puzzle: Slither to Eat

https://play.google.com/store/apps/details?id=puzzle.snakes.puzzle.slithereat.gaming

Understand the:

- Snake/worm movement
- Apple mechanics
- Growth
- Obstacles
- Puzzle structure
- Portal
- Level progression
- Difficulty
- Hint system
- UI
- Animations
- Sound
- Reward opportunities
- Advertisement flow

Then build your own original implementation.

MOST IMPORTANT REQUIREMENTS

1. GAME NAME

Must come from:

```
VITE_GAME_NAME=APPLE WORM ADVENTURE
```

and be accessed through:

```
GAME_NAME
```

---

2. SUGGESTED TECH STACK

The suggested stack is:

```
MongoDB
Express.js
React.js
Node.js
```

But:

INTERNS MAY USE THEIR PREFERRED TECH STACK.

The technology choice is flexible.

The required functionality and engineering quality are not.

---

3. VELoop ECONOMY

This is a **rewards and engagement platform**.

Therefore, every game developed for this program should think beyond gameplay.

Build meaningful:

Gameplay
↓
Engagement
↓
Rewarded Ad Opportunity
↓
Verified Ad
↓
Controlled Reward
↓
Wallet / Progress
↓
Retention

The game should monetize through **user-choice rewarded advertising**, not by overwhelming players with interruptions.

---

4. REWARDED FEATURES

At minimum, implement:

♡ Revive
↓
Rewarded Ad

🏆 Double Score
↓
Rewarded Ad

Double Coins
↓
Rewarded Ad

🎯 Extra Moves
↓
Rewarded Ad

💡 Hint
↓
Rewarded Ad

Booster

↓
Rewarded Ad

All important rewards must be:

server validated + capped + idempotent + ledgered.

---

5. ECONOMIC STABILITY

Never design the game around:

"More ads = more rewards."

Design it around:

"More meaningful engagement = more valuable ad opportunities = sustainable rewards."

The reward amounts, caps, cooldowns, and multipliers must be configurable so VELoop can tune the economy later based on actual ad revenue, retention, and user behavior.

BUILD THIS AS A REAL PRODUCT

The evaluator should not feel:

"This is just an internship CRUD project with a small game attached."

The evaluator should feel:

"This is a real VELoop game product with gameplay, engagement, monetization, rewards, analytics, and a controlled economy."

That is the expected quality bar.